

Group 3 - Space Complexity

Members:

**Kristy Mok, Ben Chalk, Corina Rivero Montiel, Euan Faller, Joel Cutler,
Luke Callen, Sam Ade Fowodu**

Change Report

6a. Continuous Integration Methods and Approaches

Our project employs a well-structured Continuous Integration (CI) approach to support seamless integration, uphold high-quality standards, and optimise development workflows. The following outlines the key components of our method and their suitability for the project:

1. **Branch-Based Workflow:**
Development is organised around feature-specific branches. Once a feature has been implemented, a pull request (PR) is created. This helps rigorous code reviews by other team members, enabling the identification of bugs and ensuring adherence to coding standards. This process not only safeguards the quality of the main branch but also fosters collaboration and knowledge sharing within the development team.
2. **Incremental Commits:**
Developers commit small, incremental changes throughout the development process. This approach ensures:
 - o **Traceability:** The ability to trace and isolate changes simplifies debugging and issue resolution.
 - o **Stability:** By allowing reversion to previous working states, when necessary, incremental commits minimise disruptions and promote stability during integration.
 - o **Efficiency:** Developers can experiment and refine features without fear of jeopardising the project's overall progress.
3. **Automated Testing and Builds:**
Using GitHub Actions, we have set up automated workflows that are triggered whenever a pull request is submitted, or changes are pushed to the main branch. This ensures that developers receive immediate feedback about integration issues, allowing for rapid resolution. Furthermore, automation guarantees consistent builds and reduces the likelihood of discrepancies between local and shared environments.
4. **GitHub Projects for Task Management:**
GitHub Projects serves as a centralised platform for task management. Tickets are created for individual features or bugs, assigned to developers, and tracked through various stages of development. This system ensures transparency, enhances accountability, and helps the team stay aligned with project milestones and priorities.
5. **Gradle and Java-Specific Configuration:**
Gradle, a versatile build automation tool, is used to manage dependencies and streamline the build process. Its efficiency and flexibility make it particularly well-suited to Java-based projects, enabling us to achieve fast, reliable builds tailored to the project's requirements.
6. **Future Plans:**
The implementation of automated testing pipelines is planned as a future enhancement. These pipelines will incorporate unit tests, code style checks, and coverage analysis to provide comprehensive quality assurance. By catching errors before they reach the main branch, this addition will reinforce the robustness of the CI process.

In conclusion, our CI approach adheres to industry best practices, promoting collaboration, keeping high standards, and easing continuous improvement as the project evolves. The scalability and reliability of this method ensure that it will continue to support the development team effectively.

6b. Continuous Integration Infrastructure Implementation

The Continuous Integration (CI) infrastructure for our project leverages GitHub Actions, providing a robust framework for automating builds, tests, and code analysis. Below is an in-depth explanation of its implementation:

1. Trigger Points:

The CI workflows are configured to activate under the following conditions:

- o **Push Events:** Whenever changes are pushed to the main branch, the pipeline is triggered to verify that the updates integrate successfully.
- o **Pull Requests:** When a pull request targeting the main branch is created, the pipeline ensures that the proposed changes meet the project's quality and compatibility standards. These triggers are essential for keeping the integrity of the codebase and minimising disruption to ongoing development.

2. Pipeline Overview:

The pipeline oversees multiple critical tasks, including:

- o Checking out the repository to access the latest code.
- o Setting up the Java development environment.
- o Running Checkstyle to enforce code style guidelines.
- o Executing unit tests and generating comprehensive reports.
- o Generating and uploading JaCoCo coverage reports.
- o Building the project and managing build artifacts, such as JAR files.

3. Implementation Details:

The CI pipeline is defined in the `.github/workflows` directory as a YAML configuration file. Below are some of the key steps included in the pipeline:

- o **Checkout and Setup:**
The repository is checked out using `actions/checkout@v4`, ensuring the latest code is accessible. JDK 17 is set up using `actions/setup-java@v4`, and Gradle is configured with `gradle/actions/setup-gradle@v4` to support efficient builds.
- o **Code Style Checks:**
Checkstyle tasks, including `'checkstyleMain'` and `'checkstyleTest'`, are executed to ensure that the code adheres to predefined style guidelines. Aggregated reports are generated and uploaded, providing actionable insights for developers to address style violations.
- o **Unit Tests and Reports:**
Unit tests are executed using Gradle. The resulting test reports are uploaded for transparency, enabling developers to verify the correctness of new features and prevent regressions.
- o **Build and Artifact Management:**
The project is built using Gradle after all checks and tests pass successfully. Build artifacts, including JAR files, are found and uploaded for future use. This step ensures that the final output is dependable and ready for deployment or distribution.

4. Future Plans:

The pipeline is designed with scalability in mind, allowing for the integration of other testing frameworks, deployment processes, and performance monitoring tools as the project evolves. These enhancements will ensure that the CI infrastructure continues to meet the demands of a growing codebase and team.

By using GitHub Actions and Gradle, our CI infrastructure delivers a comprehensive, scalable solution for keeping code quality and streamlining the development process. This implementation not only supports current project needs but also positions the team for long-term success.